sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Social Debt & Community Smells



Gemma Catolino | 2026-2027

Sustainable Software Engineering: Why Social Debt Matters in SSE

Software development is not just a technical activity; it is a long-term **collaborative process** carried out by humans embedded in social structures.
While technical debt captures the shortcuts and deficiencies in code, it fails to describe the equally important shortcuts in **communication, coordination, and collaboration**.

Sustainable Software Engineering: Why Social Debt Matters in SSE

This lecture extends the sustainability lens beyond individuals and beyond team culture into the **community layer**, where cross-team coordination, open-source governance, multi-organizational collaborations, and long-term socio-technical ecosystems operate.
A software system can only be sustainable if:

1. **Individuals** remain healthy and motivated.
2. **Teams** maintain psychological safety and shared ownership.
3. **Communities** preserve clear communication, inclusive norms, and stable collaboration networks.

Social debt emerges when the third dimension collapses.

Sustainable Software Engineering: Why Social Debt Matters in SSE

A software system can only be sustainable if:

1. **Individuals** remain healthy and motivated.
2. **Teams** maintain psychological safety and shared ownership.
3. **Communities** preserve clear communication, inclusive norms, and stable collaboration networks.

Social debt emerges when the third dimension collapses.

If left unmanaged, friction accumulates slowly: misunderstandings grow, trust decays, knowledge gets concentrated, and communication becomes fragmented.

Eventually, the socio-technical ecosystem becomes fragile — even if the code is perfect. This is why social debt is now recognized as a core component of SSE.

Sustainable Software Engineering: From Technical Debt to Social Debt: A Needed Conceptual Shift

Technical debt is visible: a messy method, a suboptimal algorithm, an outdated dependency. Social debt is invisible: a forgotten conversation, an unspoken conflict, a neglected newcomer.

Tamburri et al. (2015) defined **social debt** as the set of "accumulated sub-optimal social and organizational decisions that make collaboration harder and more costly in the future."

Examples include:

- tolerance of toxic behavior ("Known Devil"),
- reliance on one expert ("Lone Wolf"),
- permanently overloaded reviewers ("Bottleneck"),
- communication collapse between subgroups ("Silos").

Sustainable Software Engineering: From Technical Debt to Social Debt: A Needed Conceptual Shift

Technical debt is visible: a messy method, a suboptimal algorithm, an outdated dependency. Social debt is invisible: a forgotten conversation, an unspoken conflict, a neglected newcomer.

Tamburri et al. (2015) defined **social debt** as the set of "accumulated sub-optimal social and organizational decisions that make collaboration harder and more costly in the future."

Examples include:

- tolerance of toxic behavior ("Known Devil"),
- reliance on one expert ("Lone Wolf"),
- permanently overloaded reviewers ("Bottleneck"),
- communication collapse between subgroups ("Silos").

Social debt is different from technical debt in three crucial ways:

1. **It is harder to measure.**
   It does not appear in code. It appears in *behaviour*.
2. **It is harder to repair.**
   Fixing code is easier than repairing broken relationships or trust.
3. **It impacts sustainability at multiple levels simultaneously.**
   Social debt affects quality (architecture decisions), productivity (coordination), well-being (stress), and diversity (inclusion).

Sustainable Software Engineering: What Counts as Social Debt? Seeing What We Usually Ignore

Social debt includes any **structural weakness** in the human side of software engineering. Some examples:

- **Communication gaps:** unanswered questions, unclear feedback, fragmented discussions across platforms.
- **Coordination gaps:** unclear ownership boundaries, conflicting priorities, chaotic task routing.
- **Structural inequality:** some voices dominate discussions; remote contributors lose visibility.
- **Cognitive misalignment:** different assumptions about quality, deadlines, or processes.
- **Process fragility:** insufficient onboarding, unclear guidelines, inconsistent review practices.

Sustainable Software Engineering: What Counts as Social Debt? Seeing What We Usually Ignore

Social debt includes any **structural weakness** in the human side of software engineering.
Some examples:

- **Communication gaps:** unanswered questions, unclear feedback, fragmented discussions across platforms.
- **Coordination gaps:** unclear ownership boundaries, conflicting priorities, chaotic task routing.
- **Structural inequality:** some voices dominate discussions; remote contributors lose visibility.
- **Cognitive misalignment:** different assumptions about quality, deadlines, or processes.
- **Process fragility:** insufficient onboarding, unclear guidelines, inconsistent review practices.

These issues typically:

- emerge slowly,
- become normalized ("that's just how this team works"),
- create ripple effects on code quality and architecture decisions,
- accumulate until they become visible as **community smells**.

Thus, community smells are the diagnostic surface of deeper social debt.

COMMUNITY SMELLS

During Software development it might happen that some conditions...

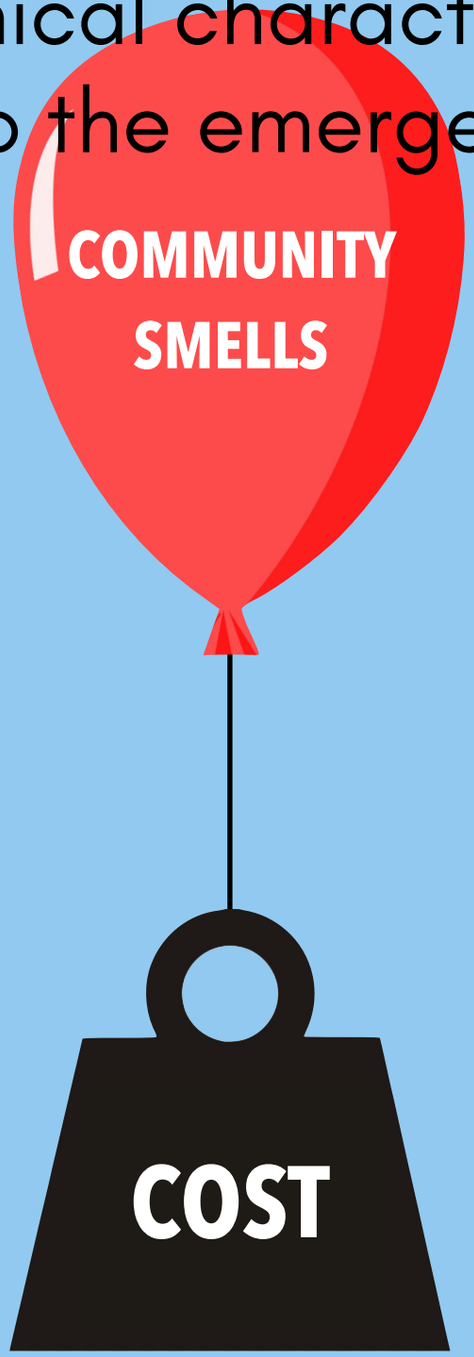


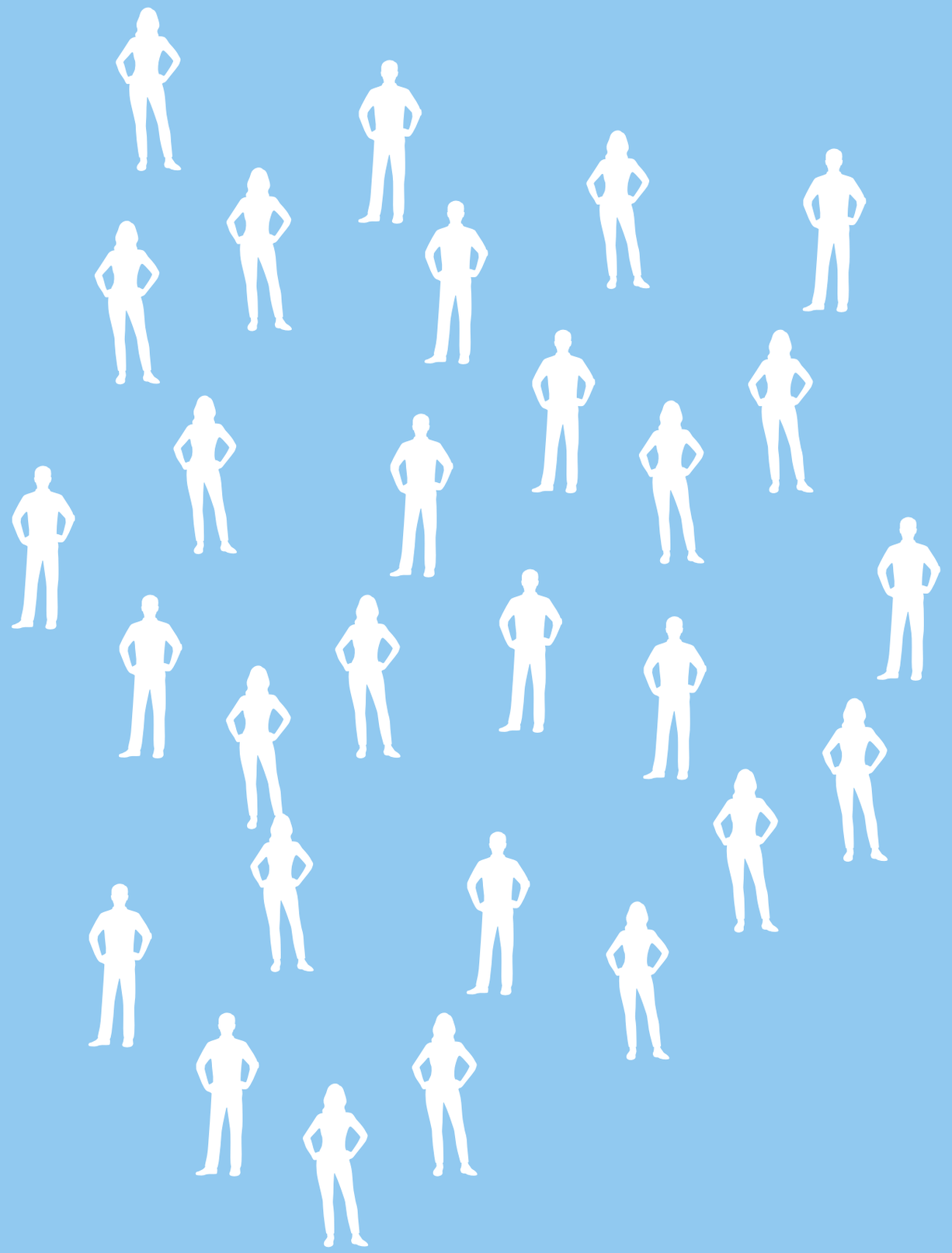
...create suboptimal organization and communication issue

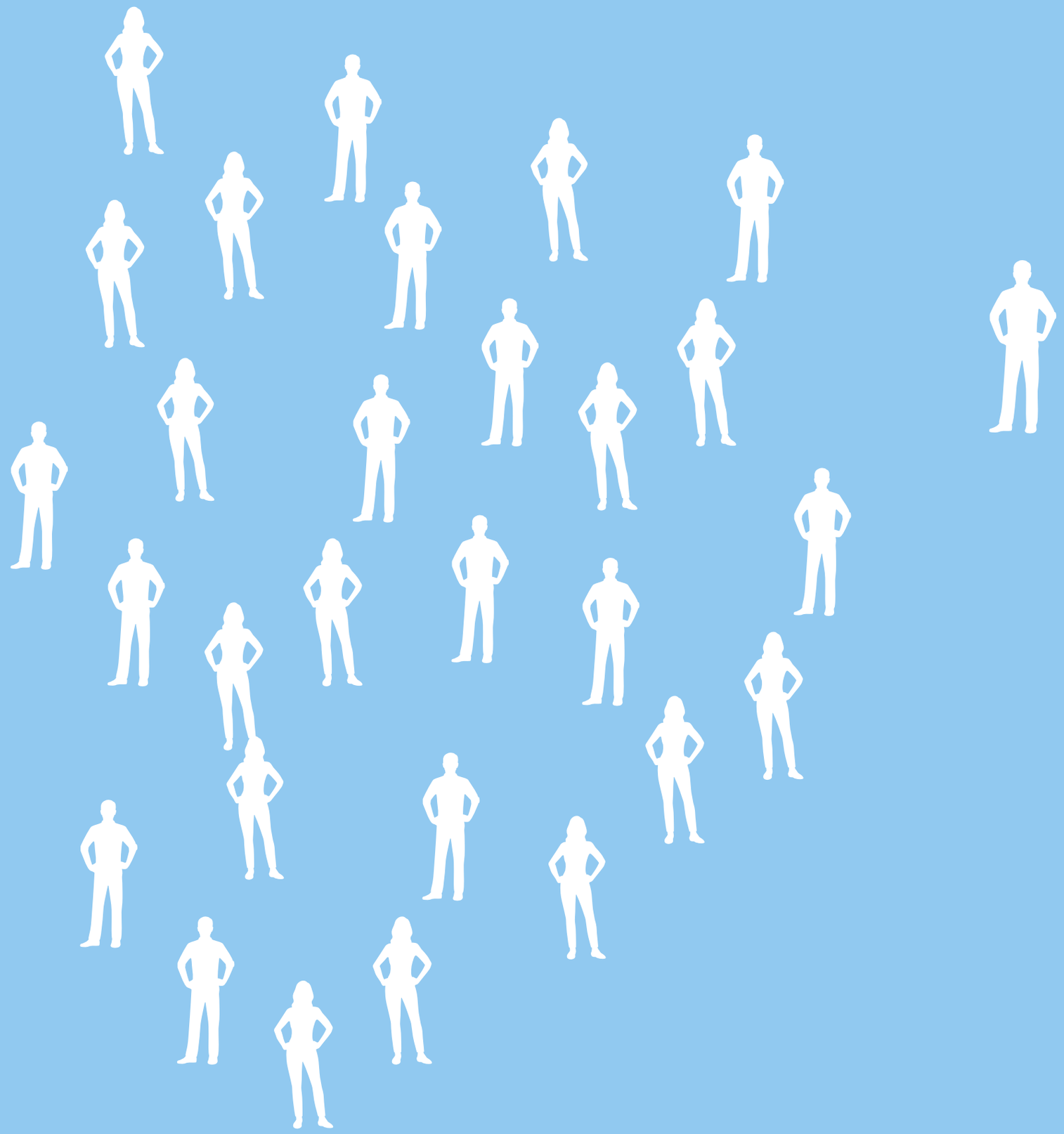


COMMUNITY SMELLS

A set of socio-technical characteristics and patterns,
which may lead to the emergence of social debt









"A contributor who carry out his/her work independently
from his/her peers, decisions, and communications"



Lone-Wolf Effect

"A contributor who carry out his/her work independently from his/her peers, decisions, and communications"

Sustainable Software Engineering: What Are Community Smells?

Community smells:

- highlight collaboration risks early,
- expose hidden dependencies in the social architecture,
- reveal behavioural anti-patterns,
- provide actionable targets for socio-technical improvement.

Analogous to code smells:

- They do **not** necessarily indicate immediate failure.
- They reflect deeper structural issues (trust, coordination, workload, power imbalance).
- They correlate with quality degradation and productivity loss.

Community smells therefore serve as **early-warning indicators** of socio-technical unsustainability.

Sustainable Software Engineering: Origins and Evolution of the Concept

1. **Early measurement attempts (Magnoni, 2015)**
   Introduced metrics for socio-technical cohesion and community structure.
2. **Social debt foundation (Tamburri et al., 2015)**
   Formalized the idea that social imperfections accumulate like technical debt.
3. **Automation and large-scale measurement**
   Provided a systematic catalog of community smells and automated detection.
4. **Knowledge gaps identification (Catolino et al., 2019, BENEVOL)**
   Reviewed the state of research and highlighted open challenges (e.g., variance, cultural factors).
5. **Industry-focused insights (Chen et al., 2024)**
   Introduced the "Known Devil" and investigated social drivers in real-world companies.
6. **Explosion of Tooling and ML Prediction**
   Detection models, network analytics, and catalogues (Almarimi 2020; Gote 2023; CADOCS 2022; Della Porta 2024) expanded the landscape.

Sustainable Software Engineering: Taxonomy of Community Smells

Community Smell	Description
Architecture by osmosis	Architecture is undocumented and shared informally.
Architecture hood	Architects are disconnected or invisible to the team.
Black cloud	Critical knowledge concentrated among a few → negative influence.
Class cognition	Code structure is hard to understand, causing cognitive overload.
Code red	Communication breaks down during critical issues.
Cognitive distance	Members think differently due to role, space, or culture, causing friction.
Cookbook development	Teams mechanically follow instructions without real comprehension.
DevOps clash	Conflict between development and operations teams.
Disengagement	Contributors withdraw or lose motivation.
Dispersion	Geographic or timezone distribution causes communication failures.
Dissensus	Frequent disagreements prevent consensus.
Hyper community	Too much communication creates noise and stress.
Informality excess	Overreliance on informal communication leads to chaotic processes.
Institutional isomorphism	Blindly copying processes from other teams or companies.
Invisible architecting	Design decisions happen without involving key collaborators.

Sustainable Software Engineering: Taxonomy of Community Smells

Community Smell	Description
Leftover techie	Senior developer avoids teaching or collaboration.
Lone wolf	A developer isolates themselves, not sharing work or knowledge.
Lonesome architecting	Architect works alone, detached from development processes.
Newbie free-riding	Newcomers rely too heavily on others without contributing proportionally.
Obfuscated architecting	Critical architectural knowledge is inaccessible or unclear.
Organizational silo	Team segmentation prevents cross-team communication.
Organizational skirmish	Conflicts or rivalries between groups or roles.
Power distance	Team hierarchy prevents equal participation in decision-making.
Priggish members	Elitist team members discourage others from participating.
Prima donnas	Contributors prioritize personal goals over collaboration.
Radio silence / Bottleneck	No response to messages or requests; progress is blocked.
Sharing villainy	Knowledge is hoarded for personal advantage.
Solution defiance	Developers reject agreed-upon solutions or instructions.
Time warp	Misaligned expectations or time usage across team members.
Unlearning	Resistance to updating old habits and practices.

ORGANIZATIONAL SILO

BLACK CLOUD

LONE WOLF

RADIO SILENCE

Community Smells



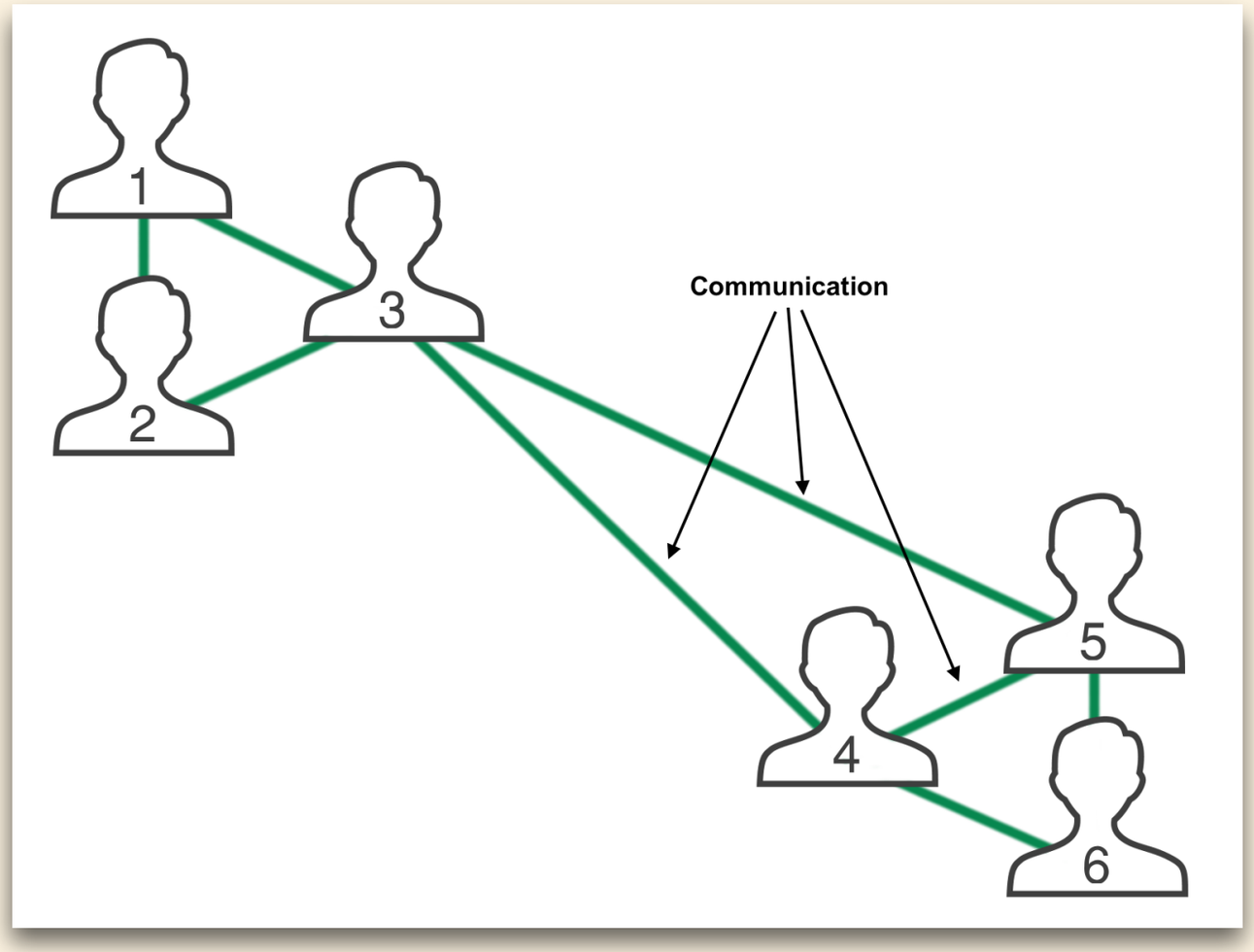
ORGANIZATIONAL SILO

This smell occurs when there are community siloed areas that do not communicate with each other, except through one or two of their respective members.

Community Smells



ORGANIZATIONAL SILO



Tamburri et al., *"The architect's role in community shepherding"*, IEEE Software, 33(6):70–79, 2016.

Community Smells



BLACK CLOUD

This smell appears when there is an excessive information overload due to the lack of structured communication.

Community Smells



RADIO SILENCE

This smell occurs when a high degree of formality in regular procedures or a specific structural organization inefficiently channels all communication through one individual or a very small group of people (a "bottleneck" or "unique boundary spanner")

Community Smells



CAUSES OF SOCIAL DEBT

Sustainable Software Engineering: Organizational Causes: When Structure Produces Friction

Organizations often generate social debt unintentionally due to:

- **Ambiguity in authority**: unclear decision makers → conflicts.
- **Unstable team composition**: rapid rotation → loss of trust, weak cohesion.
- **Competing organizational agendas**: minor disagreements turn into coordination overhead.
- **Hero cultures**: glorifying lone experts → bottlenecks, reduced collaboration.
- **Under-scoped DevOps adoption**: adopting CI/CD tools but not the cultural principles behind DevOps.

Structures shape behaviour; poor structures **force** teams into unsustainable collaboration patterns.

Tamburri et al. (2015) show that organizational anti-patterns are among the strongest drivers of social debt.

Sustainable Software Engineering: Cultural, Diversity & Human-Factor Causes

Cultural and diversity factors modulate how people communicate, disagree, escalate issues, or interpret silence.

Findings from Catolino et al. (2019, 2019 SEIS; Sarmento 2022):

- Gender-diverse teams show **less conflict escalation**,
- **Higher mentoring cohesion**,
- **More consistent communication**.

Findings from Lambiase et al. (2022, 2024):

- Cultural dispersion increases the likelihood of **misaligned expectations**,
- Geographical distance reduces **informal alignment**,
- Cultural differences in directness and hierarchy shape coordination patterns.

These are not *causes* of smells, but **amplifiers**: they magnify weaknesses in culture, process, and leadership.

Sustainable Software Engineering: Technical Causes: When Code Architecture Forces Social Architecture

Software architecture dictates who must talk to whom, and how often.

Technical drivers include:

- high coupling → more communication → more stress,
- unclear module ownership → responsibility ambiguity,
- fragile CI/CD → constant firefighting,
- lack of documentation → forced reliance on oral knowledge,
- architectural misalignment with team boundaries (Conway mismatch).

Studies in socio-technical congruence show that when **work dependencies ≠ communication dependencies**, socio-technical debt grows.

Social debt is therefore not merely "people problems":
it emerges from technical constraints, process overload, and organizational misalignment.

DETECTION, PREDICTION & TOOLING

Sustainable Software Engineering: Overview of Detection Approaches

1. Machine Learning & Data Mining

- Almarimi et al. (2020): ML models trained on issue tracker + Git + mailing list features.
- Huang et al. (2021): sentiment analysis predicts vulnerability to smell involvement.
- Neural embeddings of communication patterns (emerging work).

These models learn smell signatures from historical data to detect new instances.

Sustainable Software Engineering: Overview of Detection Approaches

1. Machine Learning & Data Mining

- Almarimi et al. (2020): ML models trained on issue tracker + Git + mailing list features.
- Huang et al. (2021): sentiment analysis predicts vulnerability to smell involvement.
- Neural embeddings of communication patterns (emerging work).

These models learn smell signatures from historical data to detect new instances.

2. Socio-Technical Metrics

Palomba & Tamburri (2021) used:

- developer centrality,
- ownership concentration,
- communication density,
- delay/latency patterns,
- cross-team dependency networks.

These metrics identify structural risks before smells fully manifest.

Sustainable Software Engineering: Overview of Detection Approaches

1. Machine Learning & Data Mining

- Almarimi et al. (2020): ML models trained on issue tracker + Git + mailing list features.
- Huang et al. (2021): sentiment analysis predicts vulnerability to smell involvement.
- Neural embeddings of communication patterns (emerging work).

These models learn smell signatures from historical data to detect new instances.

2. Socio-Technical Metrics

Palomba & Tamburri (2021) used:

- developer centrality,
- ownership concentration,
- communication density,
- delay/latency patterns,
- cross-team dependency networks.

These metrics identify structural risks before smells fully manifest.

3. Network Analytics & Higher-Order Graphs

Gote et al. (2023) extend detection with:

- higher-order centrality measures,
- triadic closure patterns,
- cross-community link failures.

Sustainable Software Engineering: Tooling Ecosystem: From Research Prototypes to Practitioner Tools

1. csDetector (Almarimi et al., 2021)

An open-source tool integrating:

- communication metrics,
- developer interaction networks,
- organizational metadata.

It produces dashboards showing hotspots of community smells, including bottlenecks, silos, and skirmishes. It is one of the first fully automated, publicly available detection systems.

2. CADOCS (Voria et al., 2022)

Focused on Slack-based communication.

The tool identifies:

- sudden communication brakes (early Radio Silence),
- power asymmetries between channels,
- repeated avoidance patterns around certain actors.

3. Community Smell Catalogue TOAST (Della Porta et al., 2024)

A structured repository containing:

- formal definitions,
- observable symptoms,
- detection rules,
- refactoring strategies.

This catalogue is essential for **standardizing terminology**, facilitating replication, and enabling scalable automation.

4. Higher-order network tools (Gote et al., 2023)

Capture communication patterns involving *triplets*, *clusters*, *and temporal dependencies*, detecting subtle forms of subgroup formation.

EFFECTS ON QUALITY, PRODUCTIVITY & WELL-BEING

Sustainable Software Engineering: How Community Smells Influence Code Quality

Social structures shape technical artifacts.

Two works (Palomba et al., ICSE 2018; TSE 2019) demonstrate that:

- Modules maintained by teams affected by community smells accumulate **higher code smell density**.
- Specific smells (Black Cloud Effect, Lone Wolf) correlate with **architectural inconsistencies**.
- Bottleneck patterns lead to **late review cycles**, which correlate with defect introduction.

Mechanism of impact:

1. Dysfunctional communication →
2. Delayed clarification and inconsistent decisions →
3. Diverging architectural interpretations →
4. Patchwork code and rushed implementations →
5. Accumulation of technical and architectural debt.

This results in *self-reinforcing cycles*:

poor collaboration → poor code → more coordination → more stress → even poorer collaboration.

Understanding this socio-technical coupling is central to sustainable software engineering.

Sustainable Software Engineering: Effects on Productivity, Throughput & Delivery

Community smells affect productivity through multiple pathways:

- **Review Latency:**
  Bottlenecks and radio silence increase time-to-merge, delaying feature delivery.
- **Coordination Overhead:**
  Silos require more cross-team synchronization, reducing usable developer time.
- **Rework Amplification:**
  Organizational skirmishes lead to misaligned implementation strategies requiring rework.
- **Decision Drift:**
  Missing or inconsistent communication leads to late discovery of misunderstandings.
- **Cognitive Load:**
  Constant negotiation in fragmented teams reduces focus on development tasks.

Evidence from Lambiase et al. (2022) shows:

- dispersed teams with unmanaged smells experience **20–40% lower throughput**,
- while teams with active smell mitigation remain stable even under dispersion.

Productivity loss from social debt is comparable to, and often worse than, technical debt.

Sustainable Software Engineering: The Human Cost of Social Debt

Community smells often manifest as:

- confusion,
- conflict,
- isolation,
- frustration.

Chen et al. (2024) found that smells like Known Devil are linked to:

- emotional exhaustion,
- psychological safety breakdowns,
- fear-based cultures.

Catolino et al. (2020) report:

- lack of ownership clarity increases stress,
- siloed groups reduce belonging,
- overloaded communicators experience burnout.

Social debt directly threatens **human sustainability** — one of the three pillars of sustainable SE (human, social, technical). A project cannot be sustainable if the **people** maintaining it are not.

VARIABILITY & DIVERSITY FACTORS

Sustainable Software Engineering: Why Smells Don't Behave the Same Everywhere

Catolino et al. (2021) identified that community smells vary according to:

- **Project Domain:**
  ML projects show more data-ownership smells; mobile apps more bottlenecks.
- **Team Size:**
  Larger communities show more Silos; smaller ones suffer Lone Wolf.
- **Governance Model:**
  Open-source communities display different smells from enterprise teams.
- **Community Maturity:**
  Early-stage communities show onboarding smells; mature communities show power asymmetries.
- **Contribution Models:**
  Volunteer communities differ from corporate-backed ones.

Social debt is not uniform — it is a **contextual phenomenon** requiring tailored detection and mitigation.

Sustainable Software Engineering: Gender Diversity: A Stabilizing Force for Community Health

Empirical studies (Catolino et al., 2019 SEIS; Catolino et al., IEEE Software 2019; Sarmento et al., 2022) consistently show that gender diversity:

- improves communication patterns and tone,
- reduces conflict escalation,
- increases mutual support and mentoring,
- enhances newcomer integration,
- correlates with lower prevalence of Silos and Skirmishes.

Diverse teams:

- encode inclusive norms,
- negotiate better,
- share responsibility more evenly.

These patterns make gender diversity not only a matter of fairness but a **socio-technical requirement for sustainability**.

Sustainable Software Engineering: Cultural & Geographical Dispersion: Amplifier, Not Cause

Lambiase et al. (2022, 2024) conducted multiple studies showing:

- dispersion intensifies smells when communication norms are unclear,
- but mitigates them when diversity is valued and structural fairness is enforced.

Key results:

- Moderate dispersion → highest creativity and cohesion.
- Minimal dispersion → fewer misunderstandings but less innovation.
- Extreme dispersion → highest smell risk (e.g., Radio Silence, Cultural Boundaries).

Cultural dimensions (directness, power distance, collectivism, uncertainty avoidance) shape:

- how people give feedback,
- how conflict is expressed,
- how help is requested,
- how leadership is interpreted.

Dispersion requires **deliberate cultural engineering** to remain sustainable.

Sustainable Software Engineering: Smell Evolution Over Time

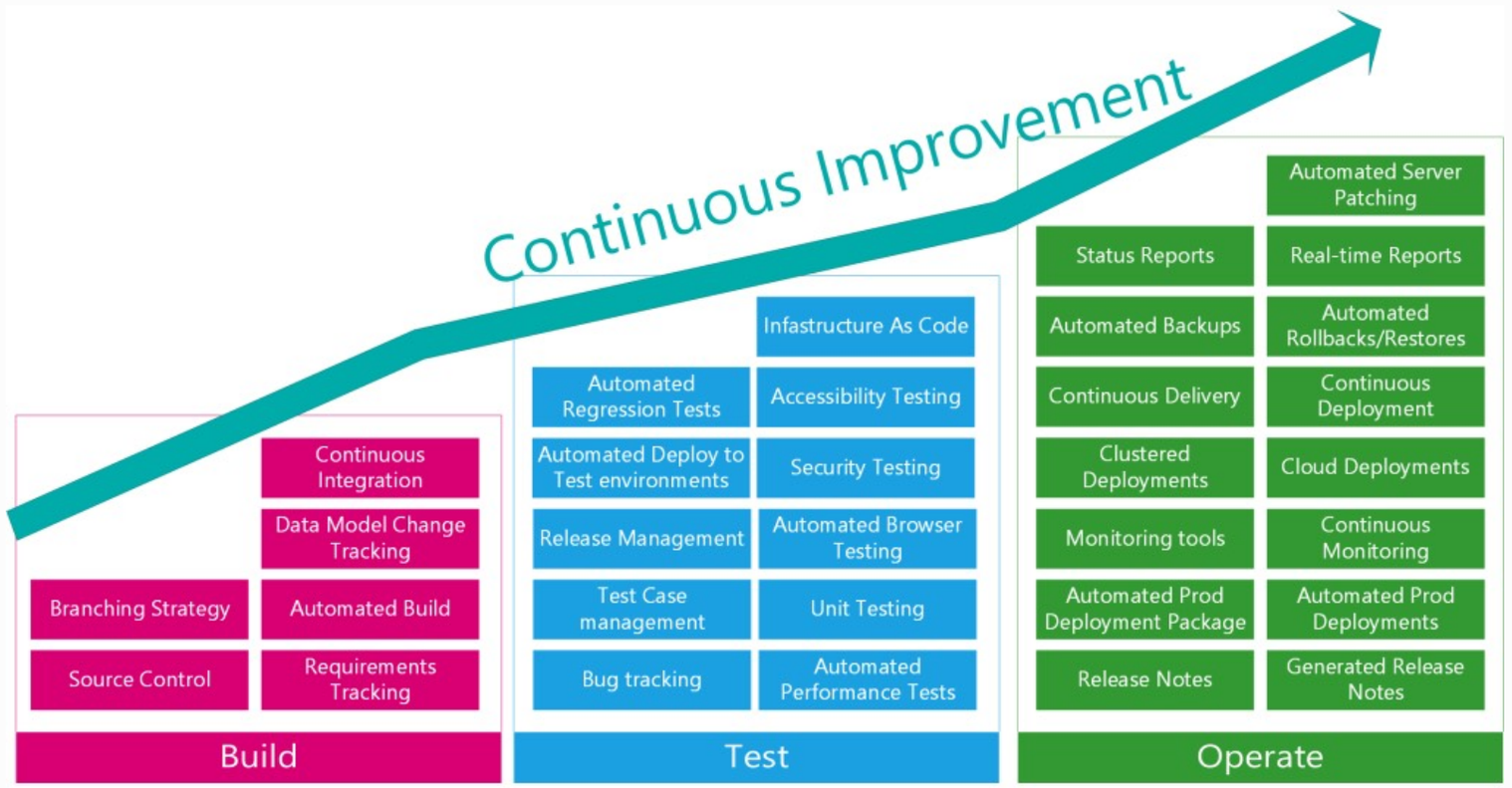
Bueno et al. (2025) performed a longitudinal analysis of smell co-occurrence:

- community smells tend to appear in **clusters**, not independently,
- some smell sequences predict others (e.g., Silos → Bottleneck),
- certain smells act as "entry points" into unsustainable cycles.

This is important because it shows:

- social debt is **temporal**,
- smells are **dynamic**,
- mitigation requires **continuous monitoring**, not one-off interventions.

This aligns with DevOps' emphasis on *continuous improvement*.



DOMAIN-SPECIFIC COMMUNITIES

Sustainable Software Engineering: ML-Enabled Software Communities

Annunziata et al. (TOSEM 2025) found that ML-enabled systems exhibit unique smells:

- **Data Gatekeeping:** data owners become bottlenecks for development.
- **Model-Feature Misalignment:** ML researchers and SE engineers misunderstand each other's priorities.
- **Experimentation Noise:** constant iterations blur communication signals.
- **Heterogeneous Expertise:** onboarding becomes significantly harder.

Cynthia et al. (2025) show:

- developers explicitly admit technical debt in ML projects when community smells occur,
- strong correlation between *metadata debt* and *seasoned gatekeepers* (data owners),
- unclear communication leads to duplication of data cleaning pipelines,
- delayed communication increases model staleness.

ML communities are thus more tightly coupled socio-technically than classical SE communities. ML ecosystems are more interdisciplinary and more fragile → **higher smell density**.

Sustainable Software Engineering: Quantum Software Communities

In quantum computing ecosystems (Lambiase et al., 2025), teams face:

- extreme expertise asymmetry (few experts → many novices),
- interdisciplinary gaps (physicists ↔ developers),
- micro-communities forming around subfields,
- deep vocabulary differences,
- onboarding barriers up to 5× higher than traditional SE.

Social debt emerges *at the birth* of these communities — making sustainability planning essential from day one.

MITIGATION & PRACTICAL STRATEGIES

Sustainable Software Engineering: Mitigating Community Smells

Effective mitigation strategies (Catolino et al., 2020; Tamburri et al., 2019):

Structural fixes:

- clarify module and code ownership,
- reassign responsibilities to break bottlenecks,
- align team boundaries with architecture (Conway alignment),
- adopt rotating leadership roles.

Process fixes:

- enforce clear communication norms,
- use decision logs,
- establish structured onboarding pipelines,
- formalize mentoring and shadowing.

Human fixes:

- conflict mediation,
- leadership coaching,
- inclusion workshops,
- blameless post-mortems.

Mitigation must operate across all layers simultaneously.

Sustainable Software Engineering: Refactoring Social Debt

The "Refactoring Community Smells in the Wild" practitioner field manual (Catolino et al., 2020) emphasizes:

- **Visibility:** social debt recedes when people "see it."
- **Dialogue:** surfacing tensions in retrospectives prevents escalation.
- **Redistribution:** spreading knowledge and responsibilities reduces smells rapidly.
- **Documentation:** social debt grows in undocumented ecosystems.
- **Small Experiments:** altering meeting formats, rotating facilitators, pairing across silos.

Refactoring social debt is not a one-time effort, but an **ongoing socio-technical practice**.

REFACTORING STRATEGIES

STRATEGY	DEFINITION
Restructure the community	Organizing sub-teams heterogeneously, splitting larger teams in smaller ones, swapping team members every task/story/sprint.
Create communication plan	Creating appropriate communication channels and protocols.
Mentoring	Talking with the team members, trying to solve communication problems.
Cohesion exercising	Doing exercises in order to improve the cohesion of the team (e.g., brainstorming, number of meetings).
Monitoring	Monitoring closely how team members communicate and share information.
Introduce a social-rewarding mechanism	Providing benefits to team members who work and communicate ap- propriately. (e.g., credits)
Not relevant	Opinions not related to refactoring strategies.

ORGANIZATIONAL SILO

REFACTORING STRATEGIES

STRATEGY	DEFINITION
Create communication plan	Creating appropriate communication channels and protocols.
Restructure the community	Organizing sub-teams heterogeneously, splitting larger teams in smaller ones, swapping team members every task/story/sprint.
Introduce a social sanctioning mechanism	Motivating people to act as expected.

REFACTORING STRATEGIES

STRATEGY	DEFINITION
Cohesion exercising	Doing exercises in order to improve the cohesion of the team (e.g., brainstorming, number of meetings).
Mentoring	Talking with the team members, trying to solve communication problems.
Create communication plan	Creating appropriate communication channels and protocols.
Restructure the community	Organizing sub-teams heterogeneously, splitting larger teams in smaller ones, swapping team members every task/story/sprint.
Monitoring	Monitoring closely how team members communicate and share information.
Introduce a social sanctioning mechanism	Motivating people to act as expected.

REFACTORING STRATEGIES

STRATEGY	DEFINITION
Mentoring	Talking with the team members, trying to solve communication problems.
Restructure the community	Organizing sub-teams heterogeneously, splitting larger teams in smaller ones, swapping team members every task/story/sprint.
Monitoring	Monitoring closely how team members communicate and share information.
Cohesion exercising	Doing exercises in order to improve the cohesion of the team (e.g., brainstorming, number of meetings).
Introduce a social sanctioning mechanism	Motivating people to act as expected.

LONE WOLF

Sustainable Software Engineering: Social Debt & Sustainability: Why It Matters for the Long Run

Social debt is not a temporary inconvenience — it's a systemic threat.
Over time, accumulated social debt manifests as:

- **Human burnout** → key contributors leave, skills are lost, onboarding gets harder.
- **Architectural drift** → design decisions erode as communication falters.
- **Productivity stagnation** → fractured communication makes everything slower.
- **Inequity and exclusion** → contributors from underrepresented groups leave first.
- **Lower resilience** → the system becomes fragile in the face of change or crisis.

From a **sustainability engineering perspective**, social debt threatens the **continuity**, **equity**, and **evolvability** of software systems — the very pillars of socio-technical sustainability.

Thus, managing social debt is as crucial as managing technical or infrastructure debt.
It ensures that **people can and want to keep contributing**, while systems remain maintainable and resilient over time.

Sustainable Software Engineering: References

Foundations & Definitions

- Magnoni, Simone. "An approach to measure community smells in software development communities." (2015).
- Tamburri, Damian A., et al. "Social debt in software engineering: insights from industry." *Journal of Internet Services and Applications* 6.1 (2015): 10.
- Tamburri, Damian A., Fabio Palomba, and Rick Kazman. "Exploring community smells in open-source: An automated approach." *IEEE Transactions on software Engineering* 47.3 (2019): 630-652.
- Chen, Hong-Mei, et al. "An empirical study of social debt in open-source projects: Social drivers and the “known devil” community smell." (2024).
- Catolino, Gemma, Fabio Palomba, and Damian A. Tamburri. "The Secret Life of Software Communities: What we Know and What we Don't Know." *BENEVOL*. 2019.
- Nazel Alshammari, Mohammed, et al. "Agile Social Software Engineer: An Empirical Approach to Mitigate and Manage Social Debts in Agile Environments." *ACM Transactions on Computing Education* 25.3 (2025): 1-23.

Detection, Prediction & Tooling

- Almarimi, Nuri, Ali Ouni, and Mohamed Wiem Mkaouer. "Learning to detect community smells in open source software projects." *Knowledge-Based Systems* 204 (2020): 106201.
- Palomba, Fabio, and Damian Andrew Tamburri. "Predicting the emergence of community smells using socio-technical metrics: A machine-learning approach." *Journal of Systems and Software* 171 (2021): 110847.
- Huang, Zijie, et al. "Predicting community smells’ occurrence on individual developers by sentiments." *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 2021.
- Gote, Christoph, et al. "Locating community smells in software development processes using higher-order network centralities." *Social Network Analysis and Mining* 13.1 (2023): 129.
- Della Porta, Antonio, et al. "A novel, tool-supported catalog of community smell symptoms." *Authorea Preprints* (2024).
- Voria, Gianmario, et al. "Community smell detection and refactoring in SLACK: The CADOCS project." *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022
- Almarimi, Nuri, et al. "csDetector: an open source tool for community smells detection." Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021.

Sustainable Software Engineering: References

Empirical Links to Code Quality & Practice

- Palomba, Fabio, et al. "How do community smells influence code smells?." *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*. 2018
- Palomba, Fabio, et al. "Beyond technical aspects: How do community smells influence the intensity of code smells?." *IEEE transactions on software engineering* 47.1 (2018): 108-129
- Catolino, Gemma, et al. "Refactoring community smells in the wild: the practitioner's field manual." *Proceedings of the acm/ieee 42nd international conference on software engineering: Software engineering in society*. 2020.
- Bueno, Arthur, et al. "Evolutionary Analysis of the Co-occurrence between Code Smells and Community Smells in Code Samples." *Simpósio Brasileiro de Sistemas de Informação (SBSI)*. SBC, 2025.

Variability, Diversity & Organizational Factors

- Catolino, Gemma, et al. "Understanding community smells variability: A statistical approach." *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 2021.
- Catolino, Gemma, et al. "Gender diversity and community smells: insights from the trenches." *IEEE Software* 37.1 (2019): 10-16.
- Catolino, Gemma, et al. "Gender diversity and women in software teams: How do they affect community smells?." *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 2019.
- Sarmento, Camila, et al. "Gender diversity and community smells: A double-replication study on brazilian software teams." *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022.
- Annunziata, Giusy, et al. "An Empirical Study on the Relation Between Programming Languages and the Emergence of Community Smells." *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2024.
- Lambiase, Stefano, et al. "Good fences make good neighbours? on the impact of cultural and geographical dispersion on community smells." *Proceedings of the 2022 ACM/IEEE 44th International Conference on Software Engineering: Software Engineering in Society*. 2022.
- Lambiase, Stefano, et al. ""There and Back Again?" on the influence of software community dispersion over productivity." *2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2022.
- Lambiase, Stefano, et al. "A Mixed-Method Empirical Investigation into the Influence of Software Communities Cultural and Geographical Dispersion Over Productivity." *Available at SSRN 4397227*.
- Lambiase, Stefano, et al. "Fright Not and Be Dispersed! Evaluating Cultural Dimensions Versus Software Communication and Collaboration Activities." *Evaluating Cultural Dimensions Versus Software Communication and Collaboration Activities*.
- van Meijel, Jari. On the relations between community patterns and smells in open-source: a taxonomic and empirical analysis. Diss. Ph. D. thesis, Eindhoven University of Technology, 2021.

Sustainable Software Engineering: References

Systematic Reviews & Syntheses

- Caballero-Espinosa, Eduardo, Jeffrey C. Carver, and Kimberly Stowers. "Community smells—The sources of social debt: A systematic literature review." *Information and Software Technology* 153 (2023): 107078.
- Suárez Brieva, Eydy del Carmen, et al. "Community smells in software development communities: systematic literature review." *Ingeniería y Competitividad* 27.2 (2025).

Domain-Specific: ML-Enabled & Data-Driven Projects

- Annunziata, Giusy, et al. "Uncovering community smells in machine learning-enabled systems: Causes, effects, and mitigation strategies." *ACM Transactions on Software Engineering and Methodology* 34.6 (2025): 1-48.
- Annunziata, Giusy, et al. "How Do Communities of ML-Enabled Systems Smell? A Cross-Sectional Study on the Prevalence of Community Smells." *arXiv preprint arXiv:2504.17419* (2025)
- Cynthia, Shamse Tasnim, Nuri Almarimi, and Banani Roy. "How Do Community Smells Influence Self-Admitted Technical Debt in Machine Learning Projects?." *arXiv preprint arXiv:2506.15884* (2025).
- Lambiase, Stefano, et al. "Socio-Technical Well-Being of Quantum Software Communities: An Overview on Community Smells." *Euromicro Conference on Software Engineering and Advanced Applications*. Cham: Springer Nature Switzerland, 2025.

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Organizational Culture and Team Well-Being



Gemma Catolino | 2026-2027